FBS: FAST BLOB STORAGE

A MINI PROJECT SYNOPSIS SUBMITTED
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Radhey Kalra (Roll Number: 2023A1R044)

Raghav Nargotra (Roll Number: 033)

Sheershak Sharma (Roll Number: 004)

SUBMITTED TO

Department of Computer Science and Engineering

Model Institute of Engineering and Technology (Autonomous)

Jammu, India

2026

CONTENTS OF SYNOPSIS

1 Project Overview 1
2 Problem Statement 1
3 Objectives of the Project 2
4 Proposed Solution / Methodology 3
5 Key Technologies and Tools 4
6 V.E.T.S Justification 4
7 Expected Outcomes 5
8 Implementation Timeline 6

1. Project Overview

Cloud object storage, led by Amazon S3, has become the standard for scalable data storage in modern computing. Virtually every cloud provider and many open-source projects implement S3-compatible interfaces, enabling portable applications that switch between backends without code changes. This project presents a lightweight, self-hosted S3-compatible storage ecosystem optimized for single-node deployments. The system consists of a high-performance Go backend with SQLite metadata, a SvelteKit web dashboard, and an OpenTUI terminal dashboard. It targets homelab enthusiasts, developers, and small teams who require S3 storage without commercial cloud dependencies or the overhead of heavyweight solutions like MinIO.

2. Problem Statement

Current self-hosted S3 solutions fail to adequately serve single-node, resource-constrained deployments. MinIO, while production-grade, requires significant memory and is architected for distributed clusters, making it impractical for homelabs on Raspberry Pi or low-end VPS instances. Simpler alternatives sacrifice S3 SDK compatibility, breaking standard AWS CLI, SDKs, and tooling that expect strict AWS signing algorithms and proper error responses. Many lack multipart upload support for large files, risking data corruption. Egress paths are unoptimized, involving excessive kernel-to-user-space copies. Additionally, administrative interfaces are tightly coupled to the storage server, offering no flexibility for users who prefer web GUI over terminal workflows.

3. Objectives of the Project

- **Design** a decoupled system architecture separating the storage engine, management API, and dashboard components to enable independent development, testing, and deployment.
- **Develop** a high-performance Go backend implementing the complete S3 API with object operations (PUT, GET, DELETE, HEAD), bucket listing (ListObjectsV2), and multipart uploads, with dual-mode authentication (AWS SigV4 and Bearer Token).
- **Implement** a SQLite-based metadata layer with WAL mode, atomic write guarantees, and an in-memory LRU cache for hot metadata queries to minimize disk I/O.
- **Optimize** data egress paths using sendfile() for zero-copy transfers, CDN-friendly caching headers, and a Go memory LRU cache to maximize throughput on commodity hardware.
- **Demonstrate** the complete ecosystem by building a SvelteKit web dashboard and an OpenTUI terminal dashboard connected to the backend via the Management API.

4. Proposed Solution / Methodology

The system follows a decoupled microservices-inspired architecture with three independent components communicating over HTTP. Figure 1 shows the complete architecture.

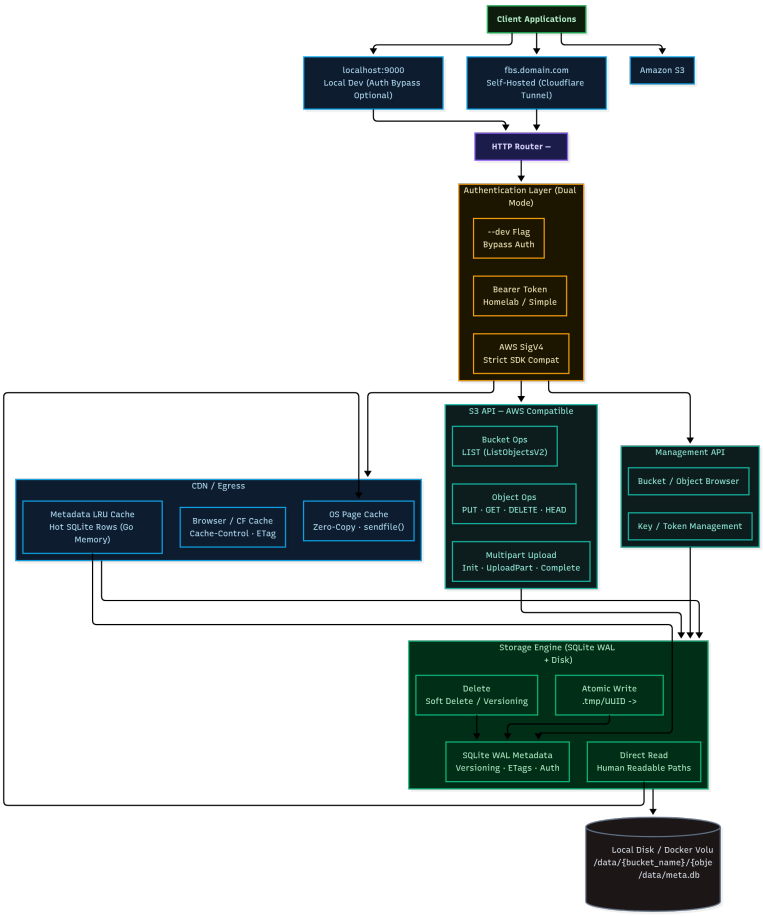

Figure 1: System Architecture of the FBS: Fast Blob Storage

The **Go backend** forms the core, handling all S3 operations and exposing two endpoint groups: the S3 API for object storage and the Management API for administration. The `chi` router handles routing with middleware for logging, panic recovery, and CORS. The storage engine writes to `.tmp/UUID` before atomic rename to `/data/{bucket}/{key}`. SQLite with WAL mode tracks metadata across four tables: `buckets`, `objects`, `multipart_uploads`, and `keys`.

The **web dashboard** (SvelteKit + TypeScript + TailwindCSS) connects via Management API, providing a metrics overview, file browser, and access key manager. The **terminal dashboard** (OpenTUI) offers a split-pane file explorer, object metadata views, and real-time metric charts with vim-style keyboard navigation.

Dual-mode authentication serves both SDK clients (AWS SigV4 signature verification) and simpler use cases (Bearer Token for dashboards and scripts). A `--dev` flag bypasses auth for local development.

5. Key Technologies and Tools

Tool	Description
Go (Golang)	Backend server implementation; goroutines for concurrency, native HTTP performance
chi Router	Lightweight HTTP routing with middleware support
SQLite (WAL Mode)	Embedded metadata storage; WAL enables concurrent reads during writes
HashiCorp LRU	In-memory cache for hot metadata queries and frequently accessed objects
SvelteKit + TypeScript + TailwindCSS	Web dashboard; file-based routing, SSR, responsive UI
OpenTUI	Terminal dashboard; declarative TUI components in TypeScript
Docker	Backend containerization with volume mounts for persistent storage

6. V.E.T.S Justification

V - Viability

The project is highly feasible within the available timeframe and resources. The Go backend compiles to a single static binary with no external runtime dependencies, deployable via a single command. SQLite requires no server process or database administration. The estimated memory footprint is under 50 MB for the backend and sub-100 MB per dashboard, making deployment viable on Raspberry Pi, low-end VPS, and container environments.

E - Engineering Depth

The project spans multiple engineering disciplines. The S3 API implementation requires deep understanding of AWS SigV4 signing (canonical request reconstruction, HMAC-SHA256), HTTP semantics, XML response formatting, and S3 error code conventions. The storage engine involves atomic file operations and directory structure management. The SQLite layer demands schema design, WAL concurrency tuning, and query optimization. Egress uses Linux `sendfile()` syscalls for zero-copy transfers.

T - Trend Alignment

The project aligns with the growing self-hosted and homelab movement driven by privacy concerns, cost optimization, and desire for infrastructure control. It follows the

cloud-native, API-first design philosophy that dominates modern distributed systems development. The TypeScript-unified stack across web and CLI tooling reflects the industry trend toward full-stack JavaScript/TypeScript ecosystems.

S - Social / Industrial Impact
Homelab enthusiasts gain an accessible, lightweight S3 target for home media servers and self-hosted application stacks. Small teams benefit from a cost-effective alternative to commercial cloud storage, eliminating per-GB egress charges while retaining S3 tooling compatibility. Educational institutions can use it as a hands-on platform for teaching distributed systems, cloud storage concepts, and API design.

7. Expected Outcomes

- A fully functional Go-based S3-compatible storage server as a drop-in replacement for AWS S3.
- Complete S3 API compliance for object CRUD, bucket listing, and multipart uploads, validated against the official AWS SDK.
- Dual authentication: AWS SigV4 verification passing official test suites, and Bearer Token auth for dashboards.
- A decoupled Management API with RESTful JSON endpoints.
- A SvelteKit web dashboard for graphical administration and file management.
- An OpenTUI terminal dashboard for keyboard-driven CLI management.
- Egress optimizations including zero-copy transfers, LRU metadata cache, and HTTP caching headers.

8. Implementation Timeline

Phase	Activity
Phase 1	Problem analysis and requirement study (Core HTTP Server & Routing, SQLite Metadata Layer, Disk Storage Engine)
Phase 2	System design and architecture (Auth Framework, Bearer Tokens, Basic S3 Object Operations)
Phase 3	Implementation and module development (AWS SigV4 Auth, Bucket Operations, Management API)
Phase 4	Testing and evaluation (Multipart Uploads, Egress Caching Optimizations, Building Web & CLI Dashboards)
Phase 5	Documentation and final presentation (End-to-end integration, Bug Fixes, Docker setup, Release)

9. References

[1] Amazon Web Services, "Amazon S3 API Reference [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html

[2] MinIO, Inc., "MinIO Object Storage [Online]. Available: https://min.io

[3] Amazon Web Services, "Authenticating Requests: Using Query Parameters (AWS Signature Version 4) [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/API/sigv4-query-string-auth.html

[4] The Go Authors, "The Go Programming Language [Online]. Available: https://go.dev

[5] Go-Chi, "chi — lightweight, idiomatic router for Go [Online]. Available: https://github.com/go-chi/chi

[6] SQLite Consortium, "Write-Ahead Logging [Online]. Available: https://www.sqlite.org/wal.html

[7] Svelte Team, "SvelteKit [Online]. Available: https://kit.svelte.dev

[8] OpenTUI, "Terminal UI Library for TypeScript [Online]. Available: https://github.com/anomalyco/opentui

[9] Linux Kernel, "sendfile(2) — transfer data between file descriptors. [Online]. Available: https://man7.org/linux/man-pages/man2/sendfile.2.html

[10] HashiCorp, "golang-lru — Golang LRU cache implementation [Online]. Available: https://github.com/hashicorp/golang-lru

[11] Deuxfleurs, "Garage — An open-source distributed object storage service [Online]. Available: https://garagehq.deuxfleurs.fr